

NumPy 선형대수 — 이번 수업용

회귀 풀이 · 고유치 분해

방금 배운 전치(`.T`) · 행렬곱(`@`) · 행렬식(`det`) · 역행렬(`inv`)이 그대로 쓰이는 두 응용을 한 자리에 모았다. 특히 직교행렬에서는 역행렬이 곧 전치($Q^{-1}=Q^T$)라는 점이, 1부의 $(X^TX)^{-1}$ 와 2부의 분해를 잇는다.

1부 · 회귀 풀이 — NumPy로 직접 푸는 선형회귀

정규방정식 $\beta=(X^TX)^{-1}X^Ty$ 와 Cramer 법칙으로 선형회귀를 직접 푼다

선형회귀의 최적 계수는 한 줄로 구해진다.

$$\beta = (X^TX)^{-1} X^Ty$$

이 한 줄에 전치(X^T), 행렬곱($X^TX \cdot X^Ty$), 역행렬($(X^TX)^{-1}$)이 모두 들어 있다. 같은 문제를 행렬식으로 푸는 Cramer 법칙도 함께 본다.

0. 문제 설정

광고비 x 에 따른 매출 y 를 직선 $\hat{y} = \beta_0 + \beta_1x$ 로 설명한다.

```
import numpy as np
x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([3, 4, 6, 5, 7], dtype=float)
```

1. 행렬로 세우기 — 설계행렬 X

절편 β_0 를 위해 1 로 채운 열을 앞에 붙인다. 그러면 $\hat{y} = X\beta$ 로 한 번에 쓸 수 있다.

```
X = np.column_stack([np.ones_like(x), x]) # 절편 열 + x 열
# X =
# [[1. 1.] [1. 2.] [1. 3.] [1. 4.] [1. 5.]] shape (5, 2)
```

2. 풀이 1 — 정규방정식 $\beta = (X^TX)^{-1}X^Ty$

전치 → 행렬곱 → 역행렬을 차례로 적용한다.

```

Xt = X.T          # (1) 전치: (5,2) → (2,5)
XtX = Xt @ X      # (2) 행렬곱: (2,5)@(5,2) = (2,2)
Xty = Xt @ y      # 행렬곱: (2,5)@(5,) = (2,)
#  $X^T X = \begin{bmatrix} 5. & 15. \\ 15. & 55. \end{bmatrix}$        $X^T y = [25. \ 84.]$ 

beta = np.linalg.inv(XtX) @ Xty      # (3)역행렬 (4)곱
#  $\beta = [2.3 \ 0.9]$  →  $\hat{y} = 2.3 + 0.9 x$ 

```

3. 풀이 2 — Cramer 법칙 (행렬식의 비율)

역행렬 대신 행렬식으로 같은 답을 얻는다. 2×2 행렬식은 $ad - bc$ 다.

$$\beta_0 = \det(M_0) / \det(X^T X), \quad \beta_1 = \det(M_1) / \det(X^T X)$$

M_0 는 $X^T X$ 의 0번 열을 $X^T y$ 로, M_1 은 1번 열을 $X^T y$ 로 바꾼 행렬이다.

```

def det2(A):          # 2x2 행렬식
    return A[0,0]*A[1,1] - A[0,1]*A[1,0]

D = det2(XtX)          # det( $X^T X$ ) = 50.0
M0 = XtX.copy(); M0[:, 0] = Xty      # 0번 열 교체
M1 = XtX.copy(); M1[:, 1] = Xty      # 1번 열 교체
b0 = det2(M0) / D; b1 = det2(M1) / D
# det( $M_0$ )=115.0 det( $M_1$ )=45.0 →  $\beta_0 = 115/50 = 2.3, \beta_1 = 45/50 = 0.9$ 

```

두 방법의 답이 $\beta = [2.3, 0.9]$ 로 같다. 행렬식이 0 이 아니므로($\det=50$) 역행렬이 존재하고, 두 풀이가 일치한다.

4. 예측과 결정계수 R^2

$$R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$$

```

yhat = X @ beta          # 예측  $\hat{y} = X\beta$ 
r2 = 1 - ((y-yhat)**2).sum() / ((y-y.mean())**2).sum()
#  $\hat{y} = [3.2 \ 4.1 \ 5.0 \ 5.9 \ 6.8]$    잔차제곱합=1.9   전체제곱합=10.0
#  $R^2 = 0.81$    (직선이 분산의 81%를 설명)

```

5. 다중회귀로 확장 — 공식은 그대로

변수가 둘(x_1, x_2)이어도 정규방정식 $\beta = (X^T X)^{-1} X^T y$ 는 그대로 쓴다. 설계행렬에 열만 하나 더 붙인다.

```

x1 = np.array([10,20,30,40,50,60], float)
x2 = np.array([ 1, 1, 2, 2, 3, 3], float)
ym = np.array([15,28,46,58,79,90], float)

```

```
Xb = np.column_stack([np.ones_like(x1), x1, x2]) # 절편 + x1 + x2, (6,3)
beta = np.linalg.inv(Xb.T @ Xb) @ Xb.T @ ym
#  $\beta = [-4.3333 \quad 1.2 \quad 7.5]$        $R^2 = 0.9994$ 
```

6. 정리 — NumPy 연산과 회귀의 대응

회귀의 한 조각	NumPy 연산
설계행렬 세우기	<code>np.column_stack([np.ones_like(x), x])</code>
X^T (전치)	<code>X.T</code>
X^TX , X^Ty (행렬곱)	<code>X.T @ X</code> , <code>X.T @ y</code>
$(X^TX)^{-1}$ (역행렬)	<code>np.linalg.inv(...)</code>
Cramer 분모·분자 (행렬식)	<code>a*d - b*c</code>
예측 $\hat{y}$	<code>X @ beta</code>

정규방정식과 Cramer 법칙은 **같은 답**을 준다. 변수가 늘어도 $\beta = (X^TX)^{-1}X^Ty$ 한 줄은 변하지 않는다. 오늘 배운 전치·행렬곱·행렬식·역행렬이 그대로 회귀의 도구가 된다.

2부 · 고유치 분해 — 대칭행렬의 분해와 이미지 압축

고유치 → 고유벡터 정규화 → $A=Q\Lambda Q^s$ 분해 → 되곱해 복원 → 이미지 압축

대칭행렬은 고유벡터가 서로 직교하므로, 단위벡터로 맞추면 직교행렬 Q (즉 $Q^{-1}=Q^T$)가 되어 $A = Q\Lambda Q^T$ 로 깔끔하게 분해된다. 큰 고유치 몇 개만 남기면 압축이 되고, 이미지가 어떻게 달라지는지 직접 본다.

1. 고유치와 고유벡터 — $A v = \lambda v$

행렬 A 를 곱해도 방향이 바뀌지 않고 크기만 λ 배 되는 벡터 v 가 고유벡터, 그 배율 λ 가 고유치다.

$$A v = \lambda v$$

```
A = np.array([[2,1],[1,2]], dtype=float)      # 대칭행렬
w, Q = np.linalg.eigh(A)    # 대칭 전용: 고유치 오름차순 + 정규직교 고유벡터
# 고유치  $\lambda = [1. \ 3.]$ 
```

2. 대칭행렬의 특별한 — 정규직교 고유벡터

대칭행렬($A=A^T$)은 고유치가 모두 실수이고 고유벡터가 서로 직교한다(스펙트럼 정리). 고유벡터를 단위벡터로 정규화하면 Q 의 열이 정규직교가 되어 Q 는 직교행렬이 된다. 직교행렬의 핵심 성질은 역행렬이 곧 전치라는 것이다.

$$Q^T Q = I \implies Q^{-1} = Q^T$$

```
# 각 고유벡터 길이 = [1. 1.]   (이미 정규화된)
#  $Q^T Q = [[1. \ 0.], [0. \ 1.]] \rightarrow$  직교행렬  $\rightarrow Q^{-1} = Q^T$ 
```

3. 고유치 분해 $A = Q \Lambda Q^T$ 와 복원

고유벡터를 열로 모은 Q 와 고유치를 대각에 놓은 Λ 로 분해한다. 직교행렬이라 Q^{-1} 대신 Q^T 를 쓴다. 거꾸로 되곱하면 원행렬이 정확히 복원된다.

$$A = Q \Lambda Q^T$$

```
recon = Q @ np.diag(w) @ Q.T
# [[2. 1.], [1. 2.]] = 원래 A (복원 성공)
```

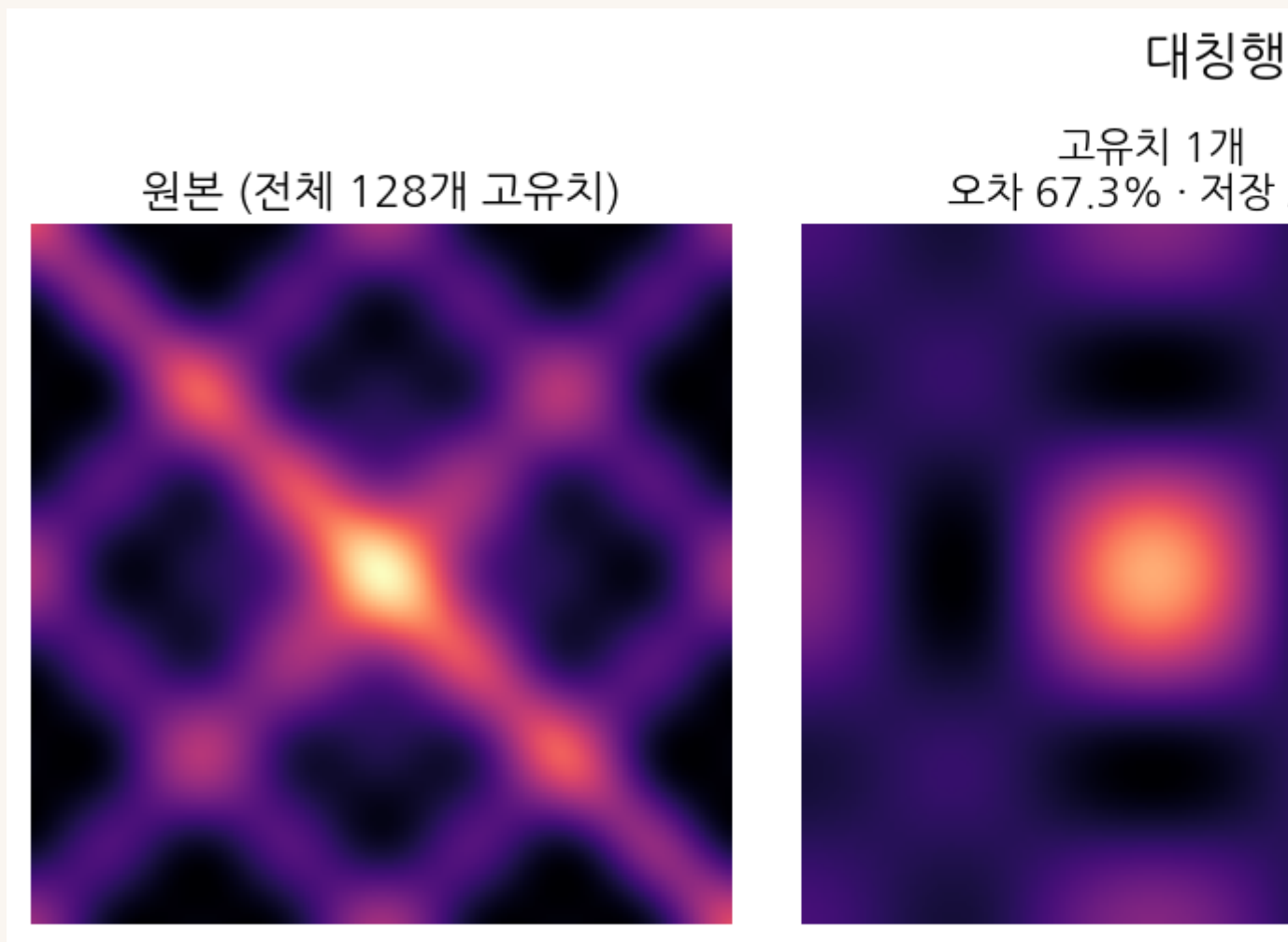
4. 스펙트럼 — 고유치별 기여의 합

고유치 분해를 풀어 쓰면 행렬은 각 고유치가 만든 랭크-1 조각들의 합이다. 큰 고유치가 큰 골격을, 작은 고유치가 세부를 담당한다.

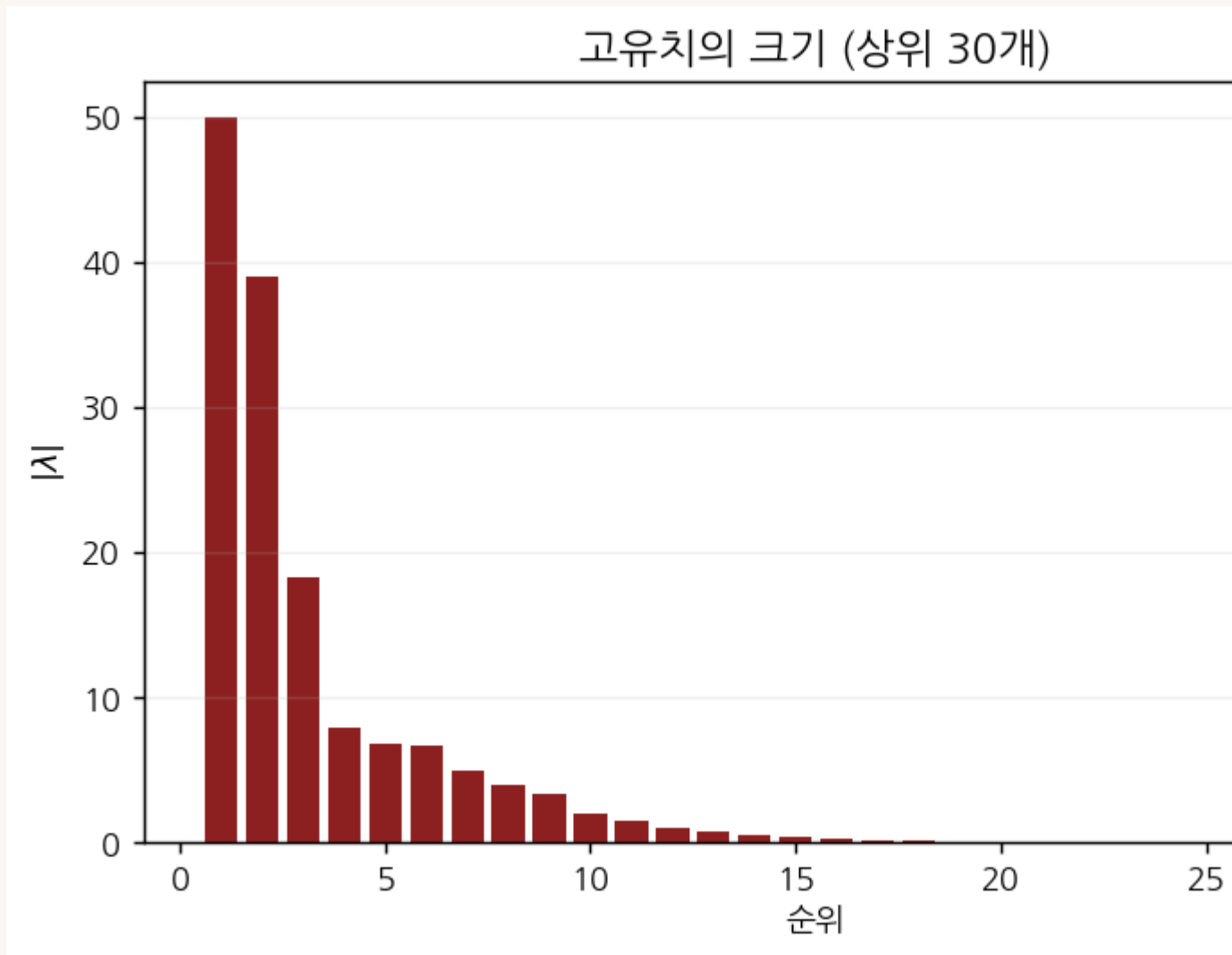
$$A = \sum \lambda_i q_i q_i^T$$

5. 이미지 압축 — 고유치 개수에 따라 이미지가 달라진다

이미지를 대칭행렬로 만들어, 큰 고유치 k 개만 남긴 랭크- k 근사 $A_k = \sum_{i \leq k} \lambda_i q_i q_i^T$ 로 복원한다.



고유치 1개는 중앙 덩어리만, 4개에서 격자가, 12개에서 거의 원본과 같아진다(오차 1.5%).



고유치 크기는 빠르게 감소하고, 12개로 누적 에너지(설명력)가 약 99.98%에 이른다.

대칭행렬에서는 고유치분해와 **SVD**(특이값 분해)가 일치한다. 일반(직사각) 이미지의 압축은 SVD로 같은 원리를 쓰며, 특이값은 대칭행렬 $A^T A$ 의 고유치의 제곱근이다.

6. 정리

단계	식 / 함수	뜻
고유치·고유벡터	$A v = \lambda v$, <code>np.linalg.eigh</code>	방향 불변, 크기만 λ 배
대칭행렬	Q 정규직교 $\rightarrow Q^{-1} = Q^T$	고유벡터 직교, 정규화하면 직교행렬
고유치분해	$A = Q \Lambda Q^T$	되곱하면 원행렬 복원
스펙트럼	$A = \sum \lambda_i q_i q_i^T$	큰 고유치가 큰 골격
압축	$A_k = \sum_{i \leq k} \lambda_i q_i q_i^T$	큰 고유치 k 개 $\rightarrow$ 근사

오늘 배운 전치·행렬곱·역행렬이 여기서도 그대로 쓰였다. 직교행렬에서 역행렬이 전치가 된다는 점이 분해를 깔끔하게 만든다.